

Command Window

New to MATLAB? See resources for [Getting Started](#).

```
>> a = fix(1/3 + 1/3 + 1/3)

a =

     1

>> b = fix(1/7 + 1/7 + 1/7 + 1/7 + 1/7 + 1/7 + 1/7)

b =

     0

>> 0.1 ^ 10000 * 0.1 ^ -10000

ans =

    NaN

>> exp(10000 * log(0.1) - 10000 * log(0.1))

ans =

     1
```

1. a與b的答案不同是因為發生了floating-point rounding error, 由於電腦用有限的位元數來儲存小數, 因此有些數值沒有辦法被精確表示, 只能用最接近的二進位數字來近似, 在 MATLAB 雙精度下, $1/3 + 1/3 + 1/3$ 的和為1, `fix()`向0取整數會得到1。但 $1/7 + 1/7 + 1/7 + 1/7 + 1/7 + 1/7 + 1/7$ 因為浮點數的加法累積誤差, 實際和為0.99999999999999978(計算相加後與1的差距, 發現比1小一個機器精度 $2.22\text{e-}16$), 因此`fix()`向0取整數會得到0, 顯示浮點數表示的極限, 在多次加法中誤差更明顯。

2. 在 $0.1^{10000} * 0.1^{-10000}$ 的運算中, 由於 0.1^{10000} 是一個極小的數, 已經遠遠小於最小數(約 $2.225\text{e-}308$)(浮點數運算是遵循IEEE 754), 因此發生了underflow, 最終結果變成0;而 0.1^{-10000} 是一個極大的數, 已經遠遠大於最大數(約 $1.797\text{e+}308$), 因此會發生overflow, 最終結果變成Inf(無窮大), 而在浮點數運算中 $0 * \text{Inf}$ 是未定義的運算, 因此Matlab會回傳NaN(非數值)。

而在 $\exp(10000 \times \log(0.1) - 10000 \times \log(0.1))$ 的運算中, $\log(0.1)$ 大約是-2.30, 因此乘上10000依舊不會underflow或overflow, 而兩數相減得到 $\exp(0) = 1$, 藉由取對數解決了溢位問題得到正確結果。